

Freight Rate Prediction Challenge

Machine Learning Validation & Model Selection Report

48,000 labeled loads | Time-aware validation | Final model: Linear Regression

Final validation result

MAE \$139.66 | RMSE \$652.00 | R^2 0.8181

Selected because it delivered the best overall holdout accuracy with the simplest and fastest pipeline.

Candidate submission report

1. Executive Summary

The goal of this assessment was to build a reliable freight-rate model that could generalize to unseen loads, not simply fit the historical data. I therefore treated the task as a forward-looking prediction problem: understand the data, clean it carefully, validate on a later month, compare several modeling approaches, investigate the remaining errors, and only then train the final model for submission.

The October holdout showed that Linear Regression was the strongest overall choice. It achieved the lowest MAE (\$139.66), the lowest RMSE (\$652.00), and the highest R^2 (0.8181) among the tested models. More complex models came close in some segments, but they did not improve the overall validation result enough to justify their added complexity and training cost.

Decision in one sentence
Based on the out-of-time validation results, I selected Linear Regression because it provided the best overall accuracy while remaining fast, transparent, and easy to reproduce.

2. Dataset and Exploratory Analysis

The labeled development dataset contained 48,000 freight loads and 14 original columns. The available inputs covered the lane, geographic coordinates, distance, equipment, weight, date, market conditions, quote signal, and the target posted_rate.

- Target distribution: posted_rate had a mean of \$2,373.98, median of \$2,030.76, and maximum of \$25,533.00. Its skewness of 1.902 showed a clear right tail.
- Distance: the strongest simple numerical predictor, with correlation 0.9085 with posted_rate.
- Equipment: average posted rates differed by type - Reefer $\approx$ \$2,553.64, Flatbed $\approx$ \$2,445.09, Dry Van $\approx$ \$2,271.55.
- Data coverage: January 1 through October 31, 2025, with no duplicate rows or duplicate load IDs.

3. Data Quality and Cleaning

The dataset was usable, but several quality issues could affect a model if left untreated. I kept the cleaning rules conservative so that valid loads were retained instead of being removed unnecessarily.

Issue	Count	Treatment	Reason
Missing weight	300	Training-median imputation	Retain otherwise valid rows
Weight <= 0	292	Set to missing, then impute	Non-positive weight is not physically meaningful
Missing market_index	374	Training-median imputation	Preserve rows without future leakage
Duplicate rows / IDs	0	No action	IDs were unique

Leakage control. Imputation values were learned from the training period only, then applied to the October holdout. This avoids using future information during model selection.

4. Validation Strategy

A random split would mix earlier and later loads and could make the evaluation look better than real future performance. I used an out-of-time split instead, training on January-September and reserving October as the holdout month.

Split	Date range	Rows	Purpose
Training	2025-01-01 to 2025-09-30	43,147	Fit preprocessing and models
October holdout	2025-10-01 to 2025-10-31	4,853	Model selection / error

			analysis
--	--	--	----------

Why this matters

The split mirrors the real use case: learn from known historical loads and predict rates for a later period. That makes the validation result more credible than a shuffled random split.

5. Feature Engineering

The feature engineering was intentionally modest. I wanted the model to gain useful structure without introducing unnecessary complexity or target leakage.

- Categorical: pickup, delivery, equipment, route.
- Numerical: distance, weight, market_index, quote_signal.
- Date-derived: month, day, day_of_week, is_weekend.
- Route: pickup_TO_delivery, representing the origin-destination pair.

For the scikit-learn models, categorical variables were one-hot encoded with unknown categories ignored at inference time. CatBoost was also evaluated with native categorical handling.

6. Model Comparison and Decision

I compared a median Dummy Regressor with four learned models using the same October holdout. The table below summarizes the results used for the final decision.

Model	MAE ↓	RMSE ↓	R ² ↑	Train time (s)
Linear Regression	139.66	652.00	0.8181	0.10
CatBoost	148.25	654.08	0.8169	81.57
Random Forest	155.09	688.69	0.7970	198.97
XGBoost	163.15	712.50	0.7827	8.12
Dummy Baseline	1,146.79	1,567.97	-0.0522	0.02

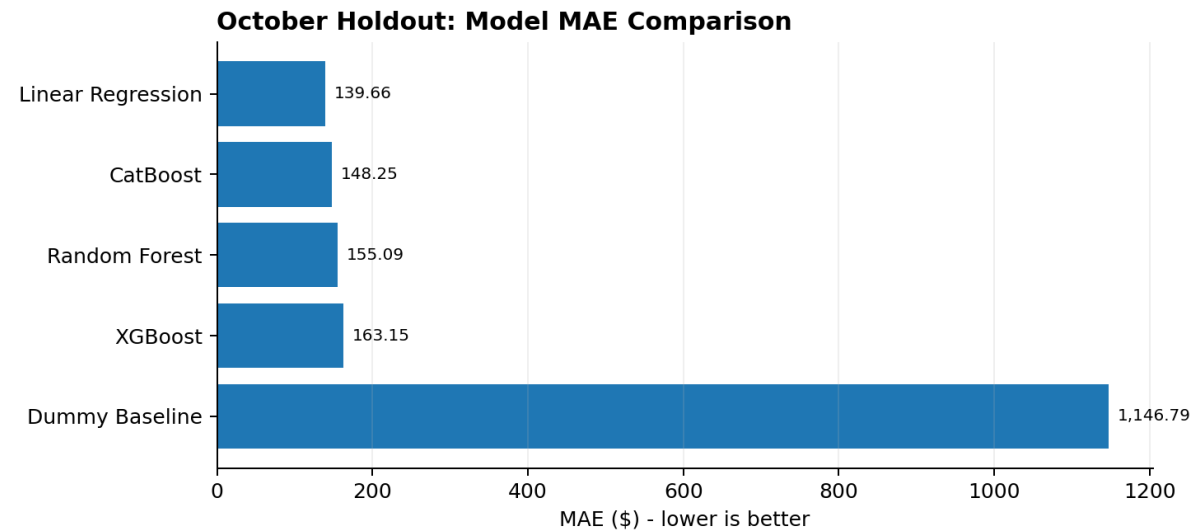


Figure 1. MAE comparison on the October out-of-time holdout.

Why Linear Regression won

It was not selected because it was the simplest model; it was selected because it was the best-performing model on the chosen validation period. It also offered a major operational advantage: near-instant training and a highly reproducible pipeline.

7. Error Analysis and Limitations

After model comparison, I inspected where the prediction errors were concentrated. This step was important because aggregate metrics alone can hide whether a model fails systematically on a particular segment.

- Linear Regression beat CatBoost on 52.92% of validation rows; CatBoost was better on 47.08%.
- For loads between 500 and 1,000 miles, Linear Regression achieved an MAE of \$86.44.
- CatBoost was stronger on Reefer loads (\$153.22 MAE vs. \$166.90 for Linear Regression), but this segment-level advantage did not change the overall ranking.
- Only 22 October loads exceeded \$7,000 (0.45% of the holdout), yet every tested regression model underpredicted all 22.

The high-value cases were not completely unseen: 150 similar loads existed in the January-September training period. Their rate-per-mile profile was also consistent across time (about \$7.53 in training and \$7.40 in validation, versus about \$2.22 for normal validation loads). The difficulty was that the available input features did not clearly identify these cases before the target was known.

To test whether a two-stage solution could help, I trained separate high-value classifiers. Random Forest improved recall but produced 1,296 false positives for only 14 correctly detected high-value loads, while Logistic Regression also had very low precision. Because the rare class could not be identified reliably, I rejected the two-stage design instead of adding complexity that would likely hurt normal-load predictions.

Main limitation

A very small number of extreme rates appear to depend on information not fully represented by the supplied features. In a production system, richer urgency, carrier-capacity, lane-condition, and real-time market features would be natural candidates to investigate.

8. Final Training and Submission Output

Once the model choice was fixed, Linear Regression was retrained on all 48,000 labeled rows. The same cleaning and feature-generation logic was then applied to data/validation.csv, and predictions were written in the required submission format.

Deliverable	validation_predictions.csv
Columns	load_id, predicted_rate
Predictions	12,000
Missing / duplicate IDs	0
Scorer status	Passed

9. December 2025 Fixed Scenario

The assessment also required a 31-day December scenario with all business inputs fixed except the date. The final model was applied to Lexington -> Fort Wayne, 360 miles, Dry Van, 32,000 lb, for every day from December 1 through December 31, 2025. The provided scorer validated all 31 rows successfully.

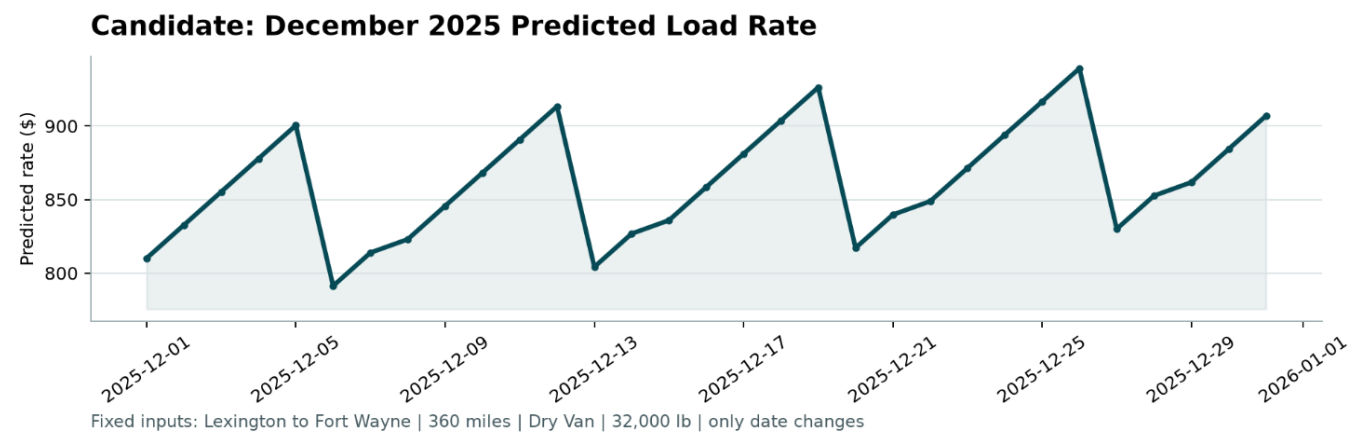


Figure 2. December 2025 predicted rate for the fixed assessment scenario (exact scorer output).

10. Reproducibility

The repository is structured so that a reviewer can follow the work from raw data through model comparison and final scoring without relying on notebook state.

- **EDA.py** - exploratory analysis and data-quality checks.
- **ETL/Extract.py** - data loading.
- **ETL/Transform.py** - cleaning, time split, imputation, and feature engineering.
- **train_model.py** - baseline and model comparison.
- **error_analysis.py** - validation error analysis.
- **train_final.py** - full-data final training and validation predictions.
- **predict_december.py** - 31-day fixed-scenario predictions.
- **score.py** - provided output validation and chart generation.

11. Final Assessment

Based on the validation results, I selected Linear Regression as the final model because it delivered the strongest overall predictive accuracy on a realistic future-month holdout. Its MAE of \$139.66, RMSE of \$652.00, and R^2 of 0.8181 were better than the more complex alternatives, while training time remained negligible.

The analysis also made clear where the solution is strong and where it is limited. For the overwhelming majority of loads, the model follows the observed pricing pattern closely. The remaining large errors are concentrated in a very small high-value tail that the supplied features do not reliably explain. I investigated that tail directly, tested an alternative classification strategy, and kept the simpler regression design when the evidence did not support added complexity.

This is the model I would submit: it is accurate on the selected holdout, easy to reproduce, transparent to explain, and validated end-to-end against the required output format. The final validation file and the December scenario both passed the provided scorer checks.